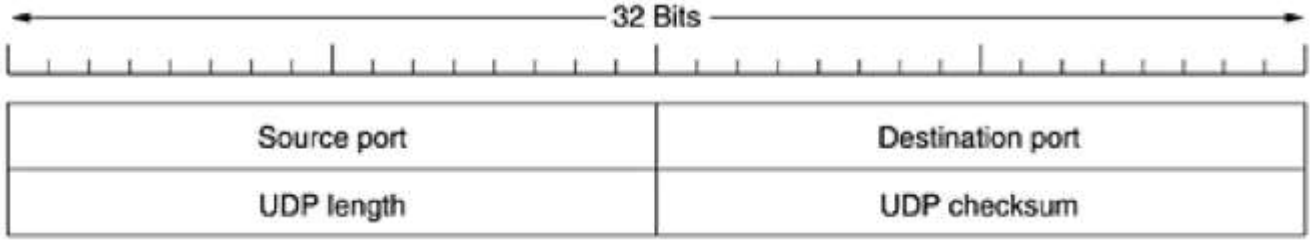
Unit II/III
Internet Transport Protocols

Topics to be discussed

- **The Internet Transport Protocols: UDP**
  - **Introduction to UDP**
  - **Remote Procedure Call**
  - **The Real-Time Transport Protocol**
- **The Internet Transport Protocols: TCP**
  - **Introduction to TCP**
  - **The TCP Service Model**
  - **The TCP Protocol**
  - **The TCP Segment Header**
  - **TCP Connection Establishment**
  - **TCP Connection Release**
  - **TCP Connection Management Modeling**
  - **TCP Transmission Policy**
  - **TCP Congestion Control**

Introduction to UDP

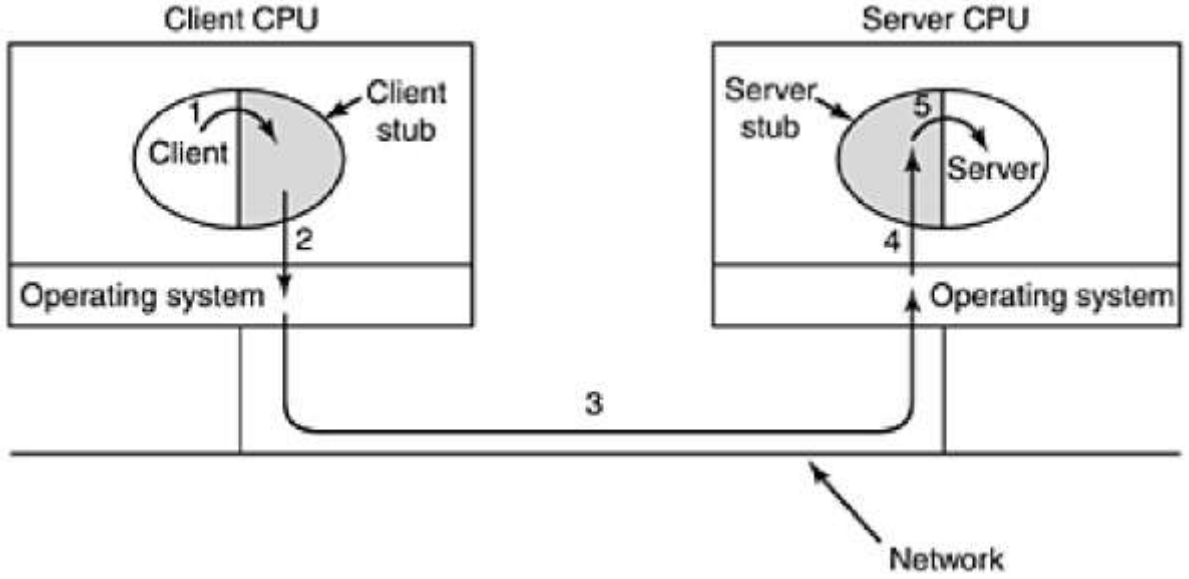
- **UDP** (**User Datagram Protocol**).
- UDP transmits **segments** consisting of an 8-byte header **followed by the payload.**



- The source port is primarily needed when a reply must be sent back to the source.
- The UDP length field includes the **8-byte header** and the data.
- It **does not do flow control, error control, or retransmission of the lost packet.**
- One area where UDP is especially useful is in **client-server situations.**
- Often, the client sends a short request to the server and expects a short reply back. If either the request or reply is lost, the client can just time out and try again.Not only is the code simple, but fewer messages are required (one in each direction).Protocol do not require an initial setup and no release is needed.
- An application that uses UDP this way is **DNS (Domain Name System)**

Remote Procedure Call

- Information can be transported from the caller to the callee in the parameters and can come back in the procedure result.
- No message passing is visible to the programmer
- This technique is known as **RPC** (**Remote Procedure Call**)
- Calling procedure is known as the **client** and the called procedure is known as the **server**
- Server is bound with a procedure called the **server stub**
- Client is bound with a procedure called the **client stub**



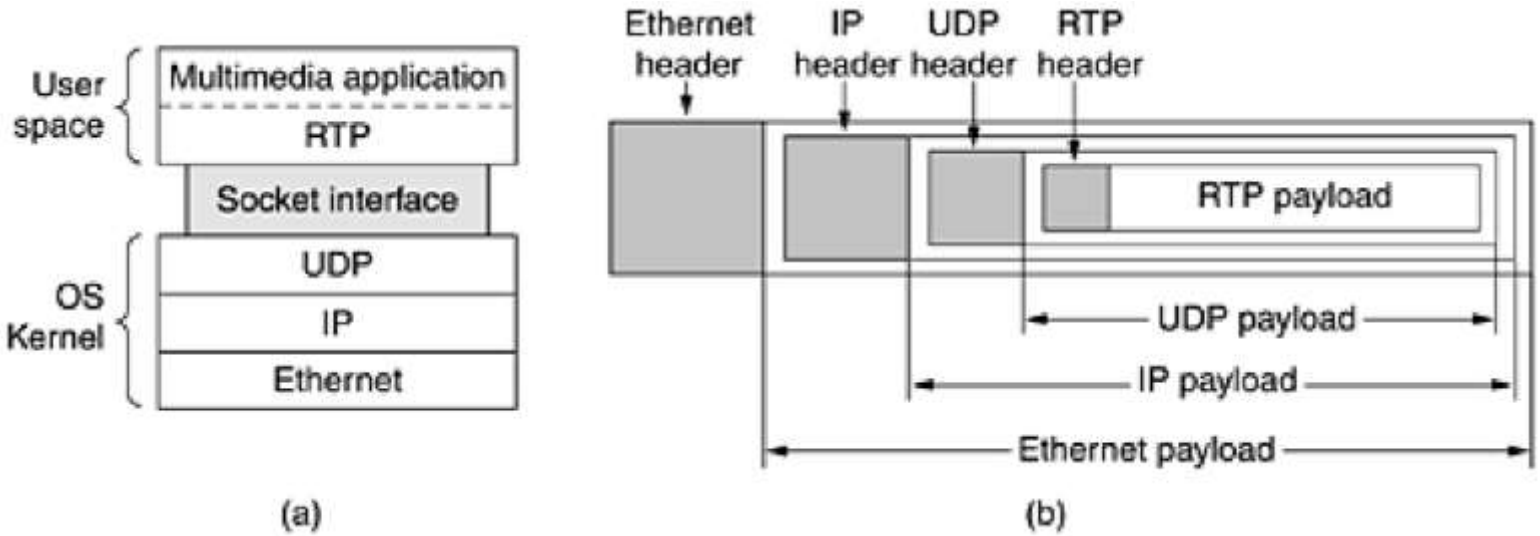
Remote Procedure Call

- Step 1 is the client calling the client stub. This call is a local procedure call, with the parameters pushed onto the stack in the normal way.
- Step 2 is the client stub packing the parameters into a message and making a system call to send the message. Packing the parameters is called **marshaling**.
- Step 3 is the kernel sending the message from the client machine to the server machine.
- Step 4 is the kernel passing the incoming packet to the **server stub**.
- Finally, step 5 is the server stub calling the server procedure with the **unmarshaled parameters**.
- **The reply traces the same path in the other direction.**
- **Some problems with RPC:**
  - Passing pointers is impossible because the client and server are in different address spaces.
  - weakly-typed languages
  - It is not always possible to deduce the types of the parameters.(An example is *printf*, which may have any number of parameters)
  - A fourth problem relates to the use of global variables. (If the called procedure is now moved to a remote machine, the code will fail because the global variables are no longer shared)
- These problems are not meant to suggest that RPC is hopeless. In fact, it is widely used, but some restrictions are needed to make it work well in practice.
- RPC need not use UDP packets, but RPC and UDP are a good fit and UDP is commonly used for RPC.

The Real-Time Transport Protocol :

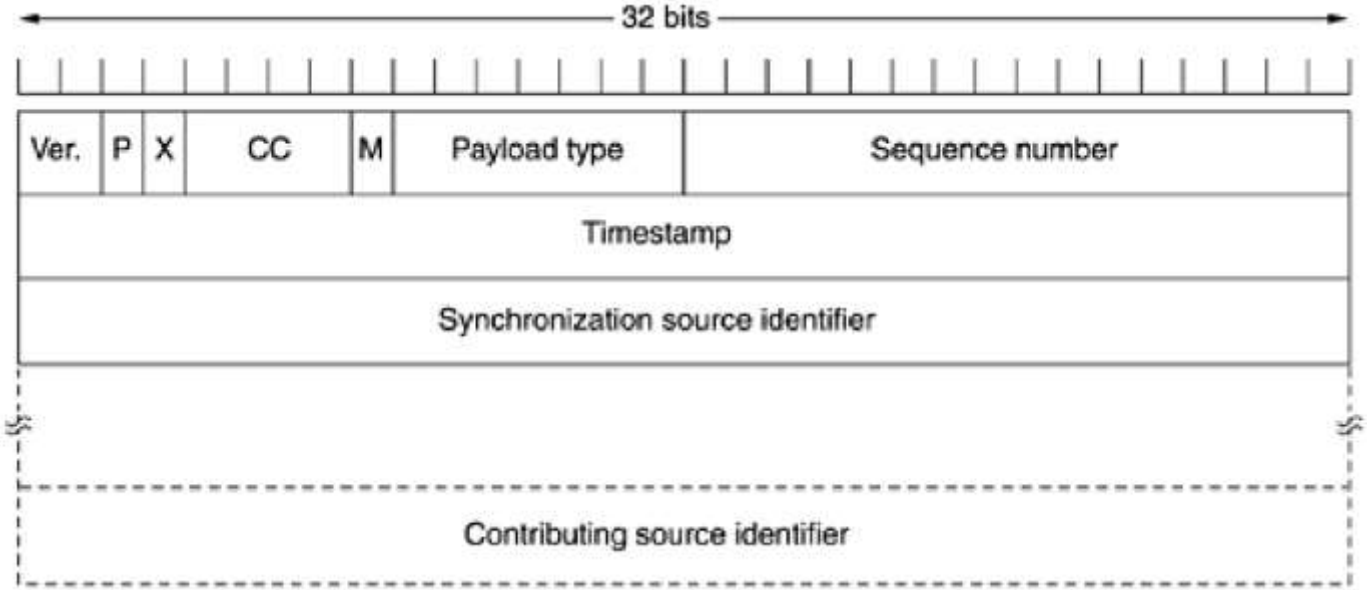
- **Udp is widely used in Client-server RPC** and real-time multimedia applications (Internet radio, Internet telephony, music-on-demand, videoconferencing, video-on-demand, and other multimedia applications)
- **RTP** (**Real-time Transport Protocol**) was developed (RFC 1889)
- It was decided to put RTP in user space and run over UDP.
- **Operation of RTP :**
  - The multimedia application consists of multiple audio, video, text, and possibly other streams.
  - These are fed into the **RTP library**, which is in user space along with the application.
  - This library then multiplexes the streams and encodes them in RTP packets, which it then stuffs into a socket.
  - At the other end of the socket (in the operating system kernel), UDP packets are generated and embedded in IP packets.

The Real-Time Transport Protocol :



- RTP looks like an application protocol as well as transport protocol.
- The basic function of RTP is to multiplex several real-time data streams onto a single stream of UDP packets.
- The UDP stream can be sent to a single destination (unicasting) or to multiple destinations (multicasting)
- Each packet sent in an RTP stream is given a number one higher than its predecessor
- Retransmission is not a practical option since the retransmitted packet would probably arrive too late to be useful.
- **RTP has no flow control, no error control, no acknowledgements, and no mechanism to request retransmissions**

The Real-Time Transport Protocol :



- It consists of three 32-bit words and potentially some extensions. The first word contains the ***Version* field**, which is already at 2
- The ***P* bit** indicates that the **packet has been padded to a multiple of 4 bytes**. The last padding byte tells how many bytes were added.
- The ***X* bit** indicates that an **extension header** is present.
- The ***CC*** field tells how many **contributing sources** are present (from 0 to 15)
- The ***M* bit** is an application-specific marker bit. It can be **used to mark the start of a video frame**, the start of a word in an audio channel, or something else that the application understands.

The Real-Time Transport Protocol :

- The ***Payload type*** field tells which **encoding algorithm** has been used (e.g., uncompressed 8-bit audio, MP3, etc.). Since every packet carries this field, the encoding can change during transmission.
- The ***Sequence number*** is just a counter that is incremented on each RTP packet sent. It is used to detect lost packets.
- The **timestamp** is produced by the stream's source to note when the first sample in the packet was made. This value can help reduce jitter at the receiver by decoupling the playback from the packet arrival time.
- The ***Synchronization source identifier*** tells which stream the packet belongs to. It is the method used to multiplex and demultiplex multiple data streams onto a single stream of UDP packets.
- Finally, the ***Contributing source identifiers***, if any, are used when mixers are present in the studio. In that case, the mixer is the synchronizing source, and the streams being mixed are listed here.
- RTP has a sister protocol called **RTCP** (**Realtime Transport Control Protocol**) : It handles feedback, synchronization, and the user interface but does not transport any data.
- RTCP also handles interstream synchronization. The problem is that different streams may use different clocks, with different granularities and different drift rates. RTCP can be used to keep them in sync.
- Finally, RTCP provides a way for naming the various sources (e.g., in ASCII text). This information can be displayed on the receiver's screen to indicate who is talking at the moment.

The Internet Transport Protocols: TCP

- **Introduction to TCP**
- **The TCP Service Model**
- **Well known ports**
- **Assembling**
- **TCP Segment Header**
- **TCP pseudo header**
- **TCP Connection Establishment**
- **TCP Connection Release**
- **TCP Connection Management Modeling**
- **TCP Transmission Policy**
- **TCP Congestion Control**

The Internet Transport Protocols: TCP

- Most of the Internet applications needs reliable, sequenced delivery.
- UDP cannot provide this, so another protocol is required; TCP

Introduction to TCP:

- TCP (Transmission Control Protocol) was specifically designed to **provide a reliable end-to-end byte stream over an unreliable internetwork.**
- TCP was designed to dynamically adapt to properties of the internetwork and to be robust
- TCP was formally defined in RFC 793, some bug fixes are detailed in RFC 1122. Extensions are given in RFC 1323.
- The IP layer gives no guarantee that datagrams will be delivered properly, so it is up to TCP to time out and retransmit them as need be.
- It is responsibility of the TCP to reassemble the packet which arrived out of order

The TCP Service Model :

- TCP service is obtained by both the sender and receiver creating end points, called sockets.
- Each socket has a socket number (address) consisting of the IP address of the host and a 16-bit number local to that host, called a **port**.
- A socket may be used for multiple connections at the same time.

TCP: Well known ports

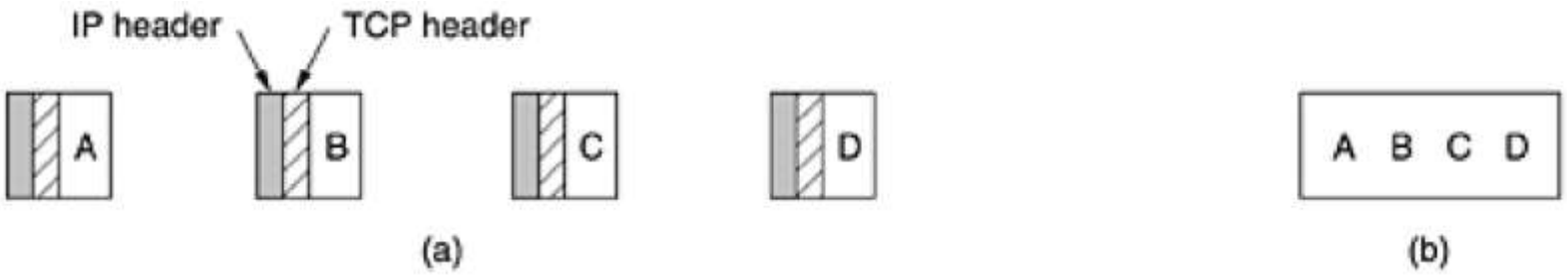
- Port numbers below 1024 are called **well-known ports** and are reserved for standard services.

  Ex :FTP can connect to the destination host's port 21.

Port	Protocol	Use
21	FTP	File transfer
23	Telnet	Remote login
25	SMTP	E-mail
69	TFTP	Trivial file transfer protocol
79	Finger	Lookup information about a user
80	HTTP	World Wide Web
110	POP-3	Remote e-mail access
119	NNTP	USENET news

- Single daemon, called **inetd** (**Internet daemon**) in UNIX, attach itself to multiple ports and wait for the first incoming connection.
- All TCP connections are full duplex and point-to-point.
- TCP does not support multicasting or broadcasting.
- A TCP connection is a byte stream, not a message stream.

TCP: Assembling

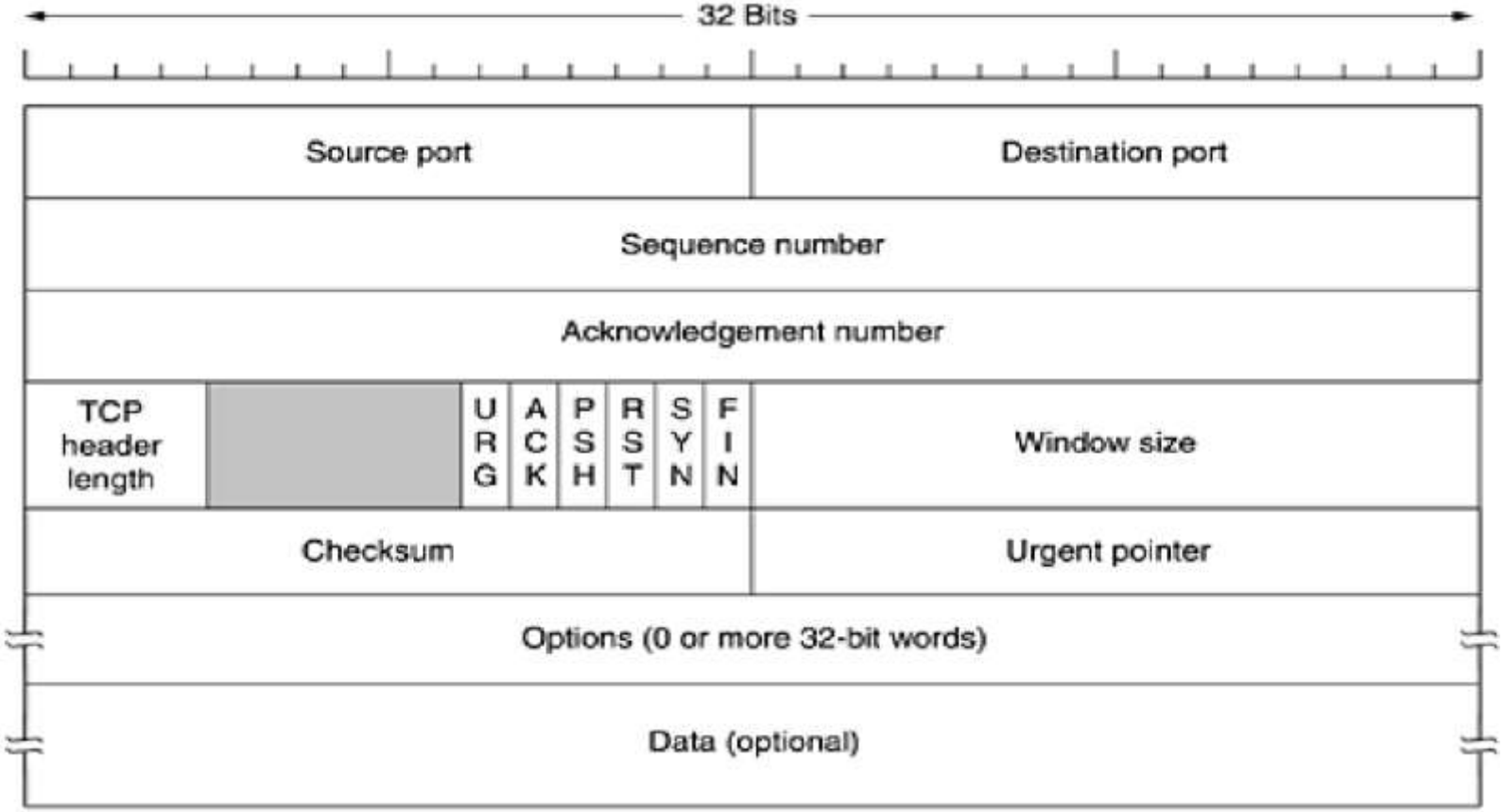


(a) Four 512-byte segments sent as separate IP datagrams. (b) The 2048 bytes of data delivered to the application in a single READ call.

- When an application passes data to TCP, TCP may send it immediately or buffer it
- The basic protocol used by TCP entities is the sliding window protocol
- Segments can arrive out of order, so bytes 3072–4095 can arrive but cannot be acknowledged because bytes 2048–-3071 have not turned up yet.
- Segments can also be delayed so long in transit that the sender times out and retransmits them.

The TCP Segment Header :

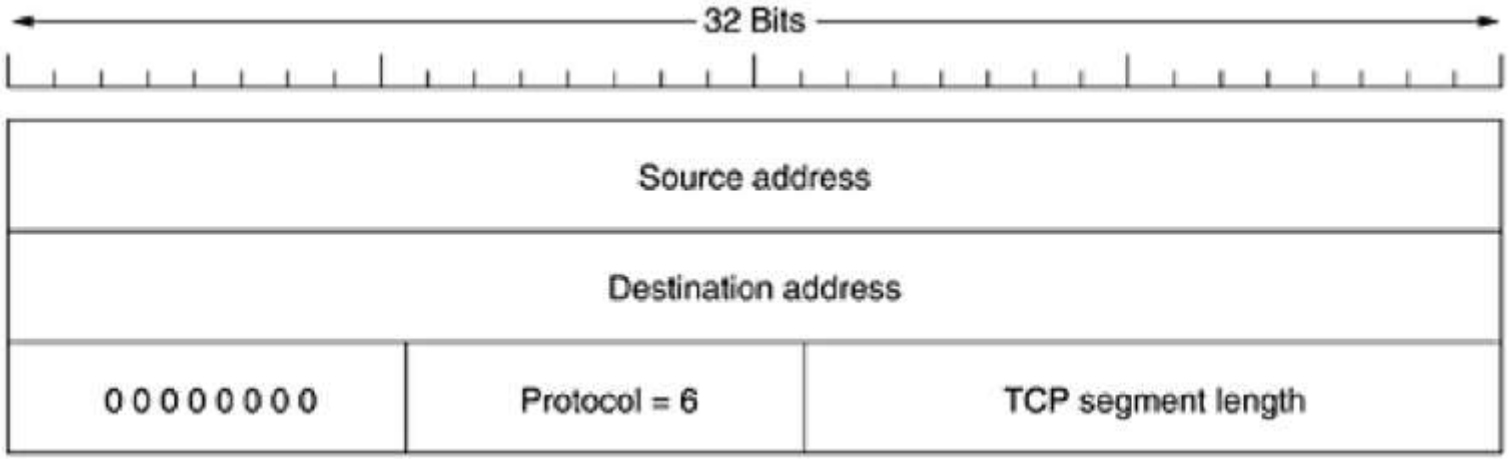
- Every segment begins with a fixed-format, 20-byte header. The fixed header may be followed by header options.
- After the option 65,535 - 20 - 20 = 65,495 data bytes



The TCP Segment Header :

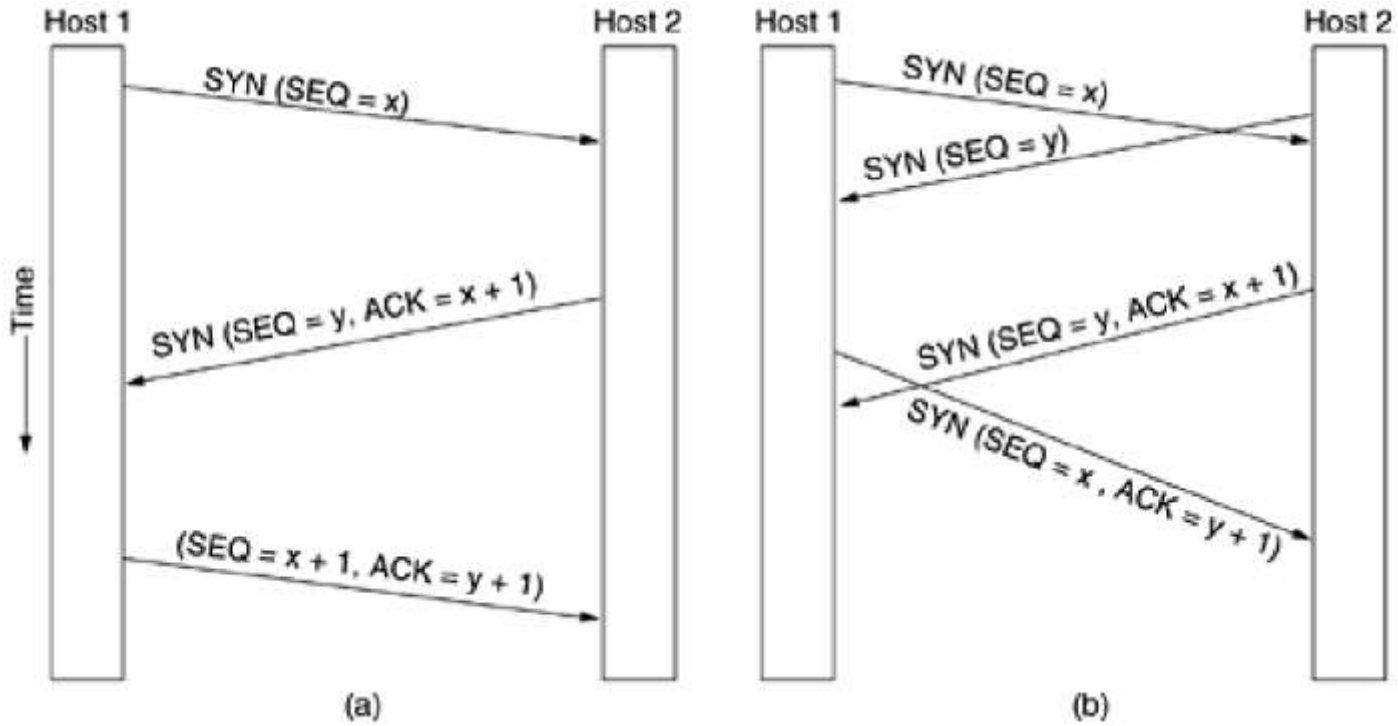
- The ***Source port*** and ***Destination port*** fields identify the local end points of the connection.
- The ***Sequence number*** and ***Acknowledgement number*** fields perform their usual functions.
- The ***TCP header length*** information is needed because the ***Options*** **field** is of variable length
- Next 6 bits are reserved.
- Now come six 1-bit flags. ***URG*** is set to 1 if the ***Urgent pointer*** is in use. The *Urgent pointer* is used to indicate a byte offset from the current sequence number at which urgent data are to be found.
- The ***ACK*** **bit** is set to 1 when transmitting an acknowledgement, 0 otherwise.
- The ***PSH*** **bit** indicates PUSHed data. The receiver is hereby kindly requested to deliver the data to the application upon arrival and not buffer it until a full buffer has been received
- The ***RST*** **bit** is used to reset a connection that has become confused due to a host crash or some other reason.
- The ***SYN*** **bit** is used to establish connections.
- The ***FYN*** **bit** is used to release connection.
- The ***Window size*** field tells how many bytes may be sent starting at the byte acknowledged.
- A ***Checksum*** is also provided for extra reliability. It checksums the header, the data, and the conceptual pseudo header

The TCP pseudo header



- Including the pseudo-header in the TCP checksum computation **helps detect misdelivered packets**.
- The ***Options* field** provides a way to add extra facilities not covered by the regular header.
  - Window scale option
  - SACK (Selective Acknowledgement)
  - TFO (TCP Fast Open)

TCP Connection Establishment :



(a) TCP connection establishment in the normal case. (b) Call collision.

- The initial sequence number on a connection is not 0
- A clock-based scheme is used, with a clock tick every 4 µsec
- when a host crashes, it may not reboot for the maximum packet lifetime to make sure that no packets from previous connections are still roaming around the Internet somewhere.

TCP Connection Establishment :

- Connections are established in TCP with **three-way handshake.**
- To establish a connection, one side, say, the server, passively waits for an incoming connection by executing the **LISTEN** and **ACCEPT** primitives.
- The other side, say, the client, executes a CONNECT primitive, specifying the IP address and port to which it wants to connect, the maximum TCP segment size it is willing to accept, and optionally some user data (e.g., a password).
- The **CONNECT** primitive sends a TCP segment with the ***SYN* bit on** and ***ACK* bit off** and waits for a response.
- When this segment arrives at the destination, the TCP entity there checks is there any process initialized LISTEN on the port mentioned in the packet in the *Destination port* field. If not, it sends a reply with the ***RST* bit on** to reject the connection.
- If some process is listening to the port, that process is given the incoming TCP segment.
- It can then either **accept** or **reject** the connection.
- If it accepts, an **acknowledgement segment** is sent back.

TCP Connection Release

- To release a connection, either party can send a TCP segment with the ***FIN* bit set**, which means that it has no more data to transmit.
- When the *FIN* is acknowledged, that direction is shut down for new data.
- Data may continue to flow indefinitely in the other direction, however.
- When **both directions have been shut down**, the **connection is released.**
- Normally, four TCP segments are needed to release a connection, **one *FIN*** and **one *ACK* for each direction. (4-way handshake)**
- However, it is possible for the **first *ACK*** and the **second *FIN*** to be contained in the same segment, reducing the **total count to three.**

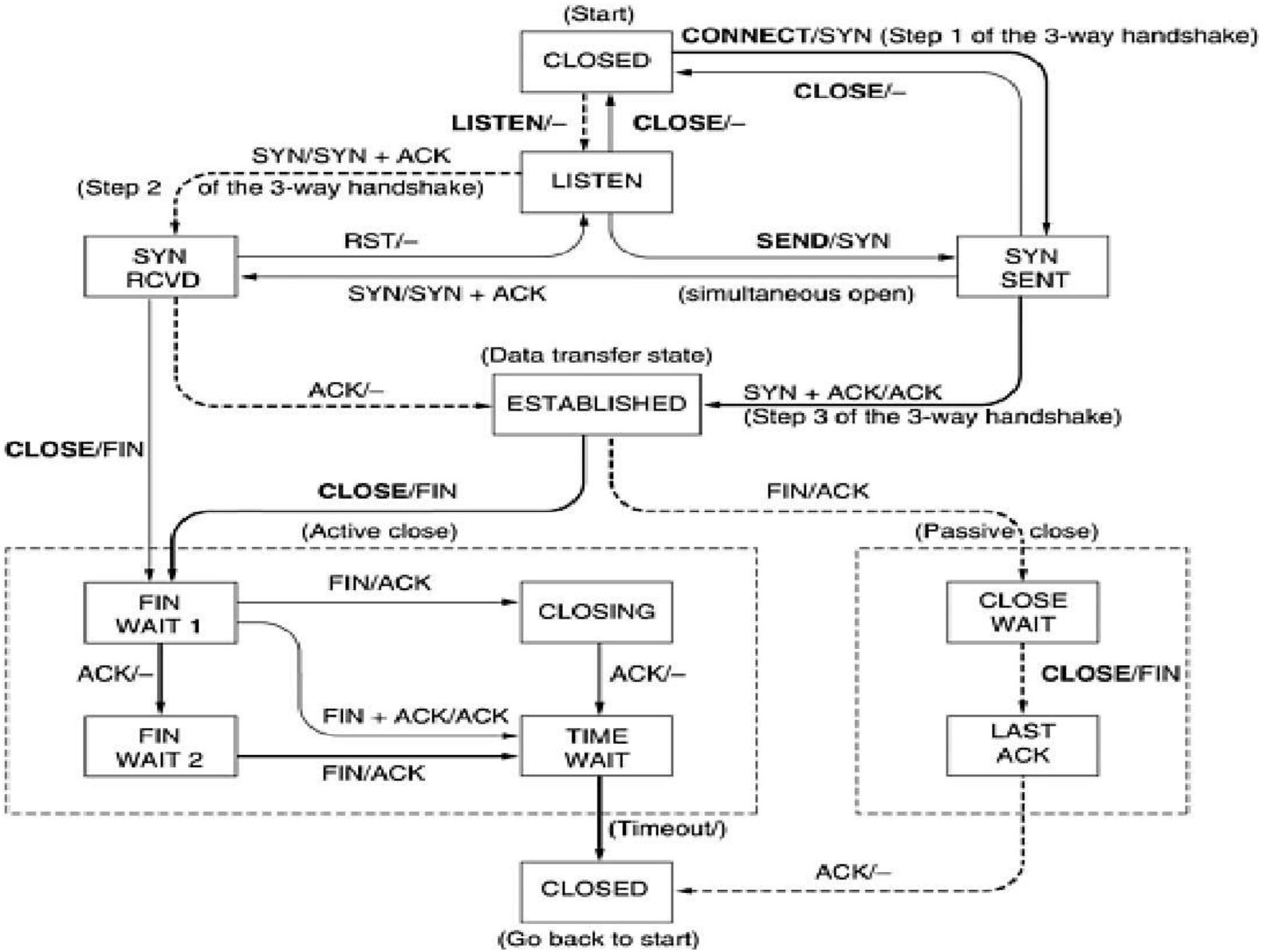
TCP Connection Management Modeling :

- The steps required to establish and release connections can be represented in a finite state machine with the 11 states.

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIMED WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

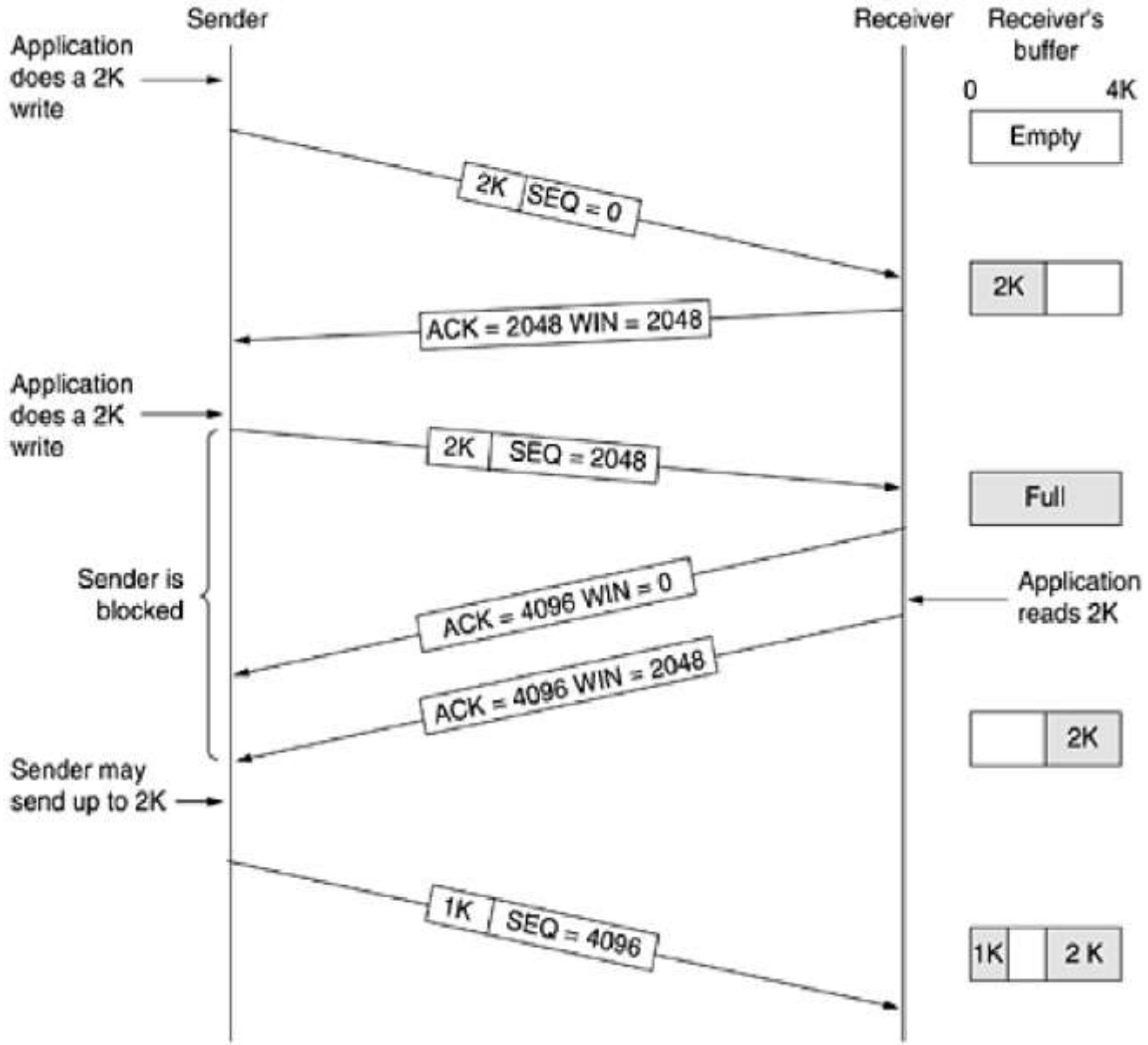
The states used in the TCP connection management finite state machine.

TCP Connection Management Modeling



- *TCP connection management finite state machine.*
- *The heavy solid line is the normal path for a client.*
- *The heavy dashed line*
- *is the normal path for a server.*
- *The light lines are unusual events.*
- *Each transition is labeled by the event causing it and the action resulting from it, separated by a slash*

TCP Transmission Policy :



Problems that can degrade TCP Performance :

1. Advertising smaller window size (due to lack of buffer space).

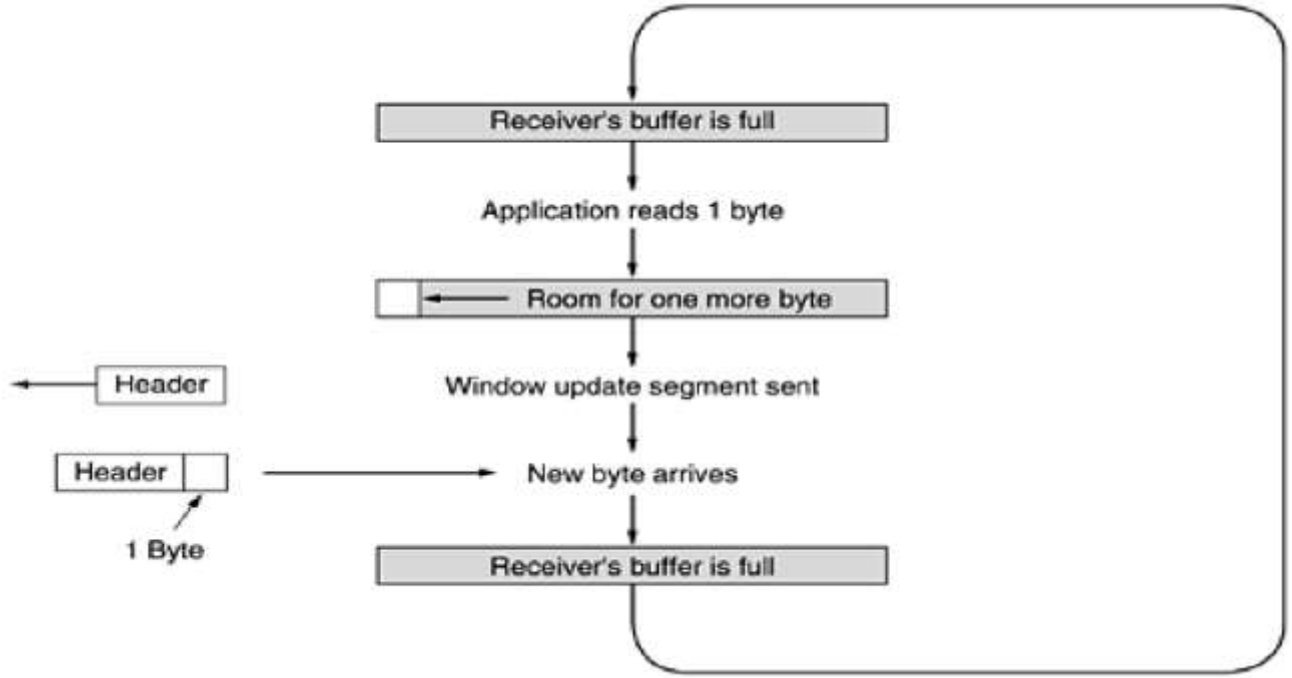
- Solutions to advertise larger window size :
  - Delay acknowledgements

2. Transmitting smaller data packets (Since application is writing slowly)

- Solution to avoid transmitting smaller data packets
  - **Nagle's algorithm :**
    - ➢ when data come into the sender one byte at a time, just send the first byte and buffer all the rest until the outstanding byte is acknowledged.
    - ➢ Then send all the buffered characters in one TCP segment and start buffering again until they are all acknowledged.
    - ➢ Nagle's algorithm is widely used by TCP implementations, but there are times when it is better to disable it.
    - ➢ Ex :when an X Windows application is being run over the Internet, mouse movements have to be sent to the remote computer. Gathering them up to send in bursts makes the mouse cursor move erratically, which makes for unhappy users.

3. **Silly window syndrome :**
   - This problem occurs when data are passed to the sending TCP entity in large blocks, but an interactive application on the receiving side reads data 1 byte at a time.

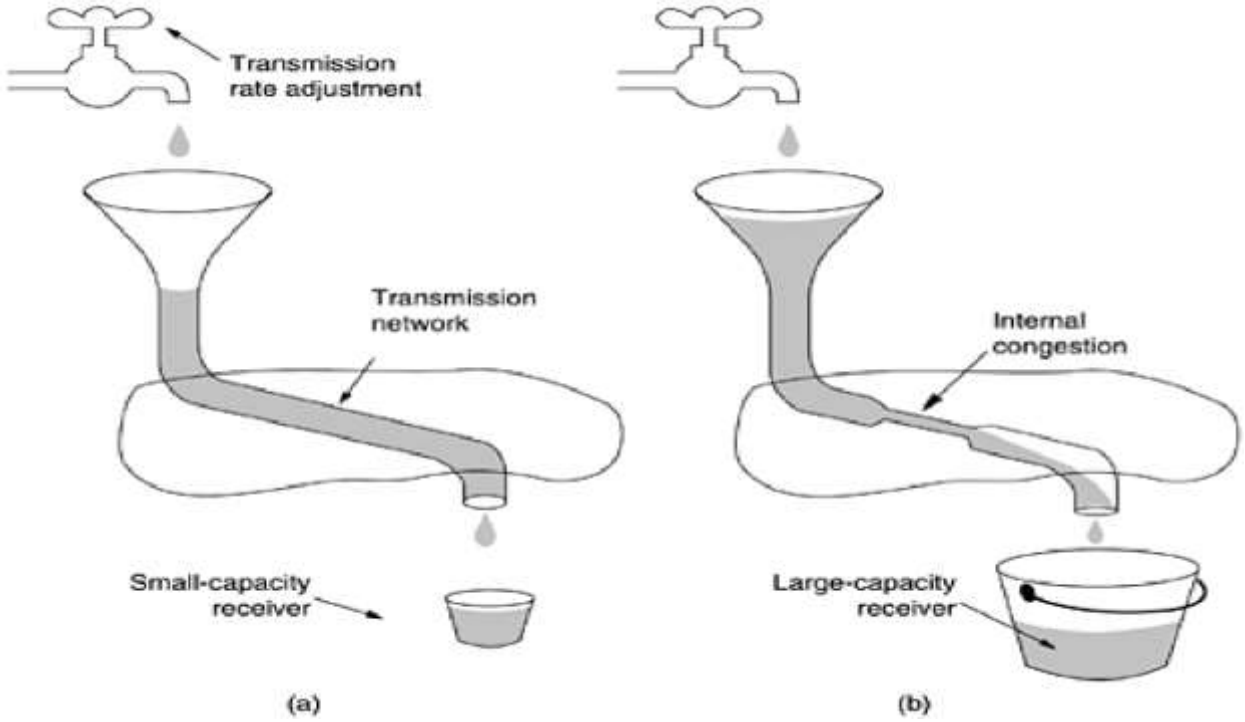
Silly window syndrome



- Solution to avoid silly window syndrome
- **Clark's solution** is to prevent the **receiver** from sending a window update for 1 byte. Instead it is forced to wait until it has a decent amount of space available (maximum segment size) and advertise.
- **Nagle's algorithm** : **sender** can also help by not sending tiny segments. Instead, it should try to wait until it has accumulated enough space in the window to send a full segment or at least one containing **half of the receiver's buffer size**
- Nagle's algorithm and Clark's solution can work together. The goal is for the sender not to send small segments and the receiver not to ask for them.
- Another receiver issue is what to do with **out-of-order segments. (Buffering the packets for reassembling also reduces the advertise window size.**

TCP Congestion Control

- When the load offered to any network is more than it can handle,it leads to **congestion**
- **The first step in managing congestion is detecting it.**
- When a connection is established, a suitable window size has to be chosen. The receiver can specify a window based on its buffer size.



(a) A fast network feeding a low-capacity receiver. (b) A slow network feeding a high-capacity receiver.

TCP Congestion Control

- Transmission should be based on: **network capacity and receiver capacity.**
- Sender need to maintain 2 window :
  - Advertise window (Receiver window)
  - Congestion window
- When a connection is established, the sender initializes the congestion window to the **size of the maximum segment** in use on the connection.
- It then sends one maximum segment. If this segment is acknowledged before the timer goes off, it adds one segment's worth of bytes to the congestion window to make it two maximum size segments and sends two segments.
- The congestion window keeps growing **exponentially** until either a timeout occurs or the receiver's window is reached.
- This algorithm is called **slow start**, but it is not slow at all (Jacobson, 1988). It is exponential.
- Internet congestion control algorithm uses **threshold.**
- When a timeout occurs, the threshold is set to half of the current congestion window, and the congestion window is reset to one maximum segment.
- Congestion window then grows exponentially until it hits the threshold. Starting then, it grows **linearly.**
- When timeout occurs , then there is a drop in congestion window and it decreases to one segment (**Additive increase multiplicative decrease AIMD)**

An example of the Internet congestion algorithm.